

Program 1:

Create a table Bank with the columns ACNO, ACT_NAME, ACT_TYPE and BAL. Specify the Primary Key. Initial BAL must be greater than 500. Write a PL/SQL program to perform debit operation by providing acct_no and amount required. The amount must be greater than 100 and less than 20000 for one transaction. If the account exists and BAL-amount>500 Bank table must be updated, otherwise “NO SUFFICIENT BALANCE” message should be displayed. If account number is not present then display “NO SUCH ACCOUNT” message to the user.

```
SQL> create table bank(  
2  accno number(15)primary key,  
3  act_name varchar2(30)not null,  
4  act_type varchar2(30)not null,  
5  bal number(10,2)check(bal>500));
```

Table created.

```
SQL> desc bank;
```

Name	Null?	Type
ACCNO	NOT NULL	NUMBER(15)
ACT_NAME	NOT NULL	VARCHAR2(30)
ACT_TYPE	NOT NULL	VARCHAR2(30)
BAL		NUMBER(10,2)

```
SQL> insert into bank values(10001,'pruthvi','savings',7000);
```

1 row created.

```
SQL> insert into bank values(10002,'anup','IT',70000);
```

1 row created.

```
SQL> insert into bank values(10003,'yunith','savings',60000);
```

```
1 row created.
```

```
SQL> insert into bank values(10004,'bhandavi','manager',50000);
```

```
1 row created.
```

```
SQL> insert into bank values(10005,'hardik','manager',70000);
```

```
1 row created.
```

```
SQL> insert into bank values(10006,'sameeksha','IT',90000);
```

```
1 row created.
```

```
SQL> insert into bank values(10007,'shreyas','savings',20000);
```

```
1 row created.
```

```
SQL> select * from bank;
```

ACCNO	ACT_NAME	ACT_TYPE	BAL
10001	pruthvi	savings	7000
10002	anup	IT	70000
10003	yunith	savings	60000
10004	bhandavi	manager	50000
10005	hardik	manager	70000
10006	sameeksha	IT	90000
10007	shreyas	savings	20000

```
7 rows selected.
```

```
SQL> ed Bank.sql
```

```
set serveroutput ON;
```

```
ACCEPT acno NUMBER PROMPT "Enter the Account Number:"
```

```
ACCEPT amount1 NUMBER PROMPT "Enter the Amount to Withdraw:"
```

```
DECLARE
```

```
    acno number(10);
```

```
    amount1 number(10);
```

```

        db_accno number(10);

        db_amount number(10);

        db_name varchar(25);

BEGIN

        acno:=&acno;

        amount1:=&amount1;

        SELECT accno,bal INTO db_accno,db_amount

        FROM bank

        WHERE accno = acno;

IF (amount1>100 AND amount1<20000) then

        IF db_amount>100 then

                UPDATE bank

                SET bal=bal-amount1

                WHERE accno=acno;

                COMMIT;

                Dbms_output.put_line('WITHDRAWAL SUCCESS. ');

                SELECT accno,bal,act_name INTO db_accno,db_amount,db_name

                FROM bank

                WHERE accno = acno;

                DBMS_OUTPUT.PUT_LINE ('Available Balance: '|| db_amount);

                DBMS_OUTPUT.PUT_LINE ('Name: '|| db_name);

                DBMS_OUTPUT.PUT_LINE ('Account Number: '|| db_accno);

        ELSE

                Dbms_output.put_line('NO SUFFICIENT BALANCE!');

        END IF;

ELSE

        Dbms_output.put_line('INVALID AMOUNT');

END IF;

EXCEPTION

```

```
WHEN no_data_found THEN
```

```
    dbms_output.put_line('NO SUCH ACCOUNT!');
```

```
END;
```

```
/
```

OUTPUT:

```
set serveroutput ON;
```

```
SQL> @"C:\Users\divya\OneDrive\Pictures\Saved Pictures\bank.sql";
```

```
Enter the Account Number:101
```

```
Enter the Amount to Withdraw:2000
```

```
old 8:      acno:=&acno;
```

```
new 8:      acno:= 101;
```

```
old 9:      amount1:=&amount1;
```

```
new 9:      amount1:= 2000;
```

NO SUCH ACCOUNT!

PL/SQL procedure successfully completed.

```
SQL> @"C:\Users\divya\OneDrive\Pictures\Saved Pictures\bank.sql";
```

```
Enter the Account Number:10001
```

```
Enter the Amount to Withdraw:200
```

```
old 8:      acno:=&acno;
```

```
new 8:      acno:= 10001;
```

```
old 9:      amount1:=&amount1;
```

```
new 9:      amount1:= 200;
```

WITHDRAWAL SUCCESS.

```
Available Balance: 6800
```

```
Name: pruthvi
```

```
Account Number: 10001
```

PL/SQL procedure successfully completed.

SQL> @"C:\Users\divya\OneDrive\Pictures\Saved Pictures\bank.sql";

Enter the Account Number:10006

Enter the Amount to Withdraw:900000

old 8: acno:=&acno;

new 8: acno:= 10006;

old 9: amount1:=&amount1;

new 9: amount1:= 900000;

INVALID AMOUNT

PL/SQL procedure successfully completed.

Program 2:

Create a table STOCK_DETAIL with the columns PNO, PNAME and QTY_AVL to store stock details of computer accessories. Specify Primary Key and NOT NULL constraints on the table. QTY_AVL should be positive number. Write a PL/SQL Program to define a user defined exception named "LOW_STOCK" to validate the transaction. The program facilitates the user to purchase the product by providing product number and quantity required. It should display an error message "NO SUFFICIENT STOCK" when the user tries to purchase a product with quantity more than QTY_AVL, Otherwise the STOCK_DETAIL table should be updated for valid transaction.

```
SQL> create table stock_detail(  
2  pno varchar2(10) primary key,  
3  pname varchar2(20) not null,  
4  qty_avl number(30)check(qty_avl>0));
```

Table created.

```
SQL> desc stock_detail;
```

Name	Null?	Type

PNO	NOT NULL	VARCHAR2(10)
PNAME	NOT NULL	VARCHAR2(20)
QTY_AVL		NUMBER(30)

```
SQL> insert into stock_detail values('101','computer',50);
```

1 row created.

```
SQL> insert into stock_detail values('102','keyboard',30);
```

1 row created.

```
SQL> insert into stock_detail values('103','mouse',50);
```

1 row created.

```
SQL> insert into stock_detail values('104','cpu',60);
```

```
1 row created.
```

```
SQL> insert into stock_detail values('105','pendrive',80);
```

```
1 row created.
```

```
SQL> ed stock_detail.sql
```

```
set serveroutput ON;
```

```
ACCEPT prdtNo CHAR PROMPT "Enter the Product Number:"
```

```
ACCEPT Qty NUMBER PROMPT "Enter the Quantity Required:"
```

```
DECLARE
```

```
prdtNo varchar2(10);
```

```
prodName varchar2(25);
```

```
Qty number(10);
```

```
db_prodNo varchar2(10);
```

```
db_prodQty number(10);
```

```
LOW_STOCK EXCEPTION;
```

```
BEGIN
```

```
prdtNo:='&prdtNo';
```

```
Qty:='&Qty';
```

```
SELECT pno,pname,qty_avl INTO db_prodNo,prodName,db_prodQty
```

```
FROM stock_detail
```

```
WHERE pno = prdtNo;
```

```
DBMS_OUTPUT.PUT_LINE ('Available Stock: '|| db_prodQty);
```

```
DBMS_OUTPUT.PUT_LINE ('Product Name: '|| prodName);
```

```
IF (Qty<db_prodQty) then
```

```
UPDATE stock_detail
```

```
SET qty_avl=qty_avl-Qty
```

```
WHERE pno=prdtNo;
```

```
COMMIT;
```

```

Dbms_output.put_line('Values Updated');

SELECT pno,pname,qty_avl INTO db_prodNo,prodName,db_prodQty
FROM stock_detail

WHERE pno = prdtNo;

DBMS_OUTPUT.PUT_LINE ('Available Stock: '|| db_prodQty);

DBMS_OUTPUT.PUT_LINE ('Product Name: '|| prodName);

ELSE

raise LOW_STOCK;

END IF;

EXCEPTION

WHEN no_data_found THEN

dbms_output.put_line('NO SUCH PRODUCT!');

WHEN LOW_STOCK THEN

dbms_output.put_line('NO SUFFICIENT STOCK!');

END;

/

```

OUTPUT:

```

SQL> set serveroutput ON;

SQL> @"C:\Users\divya\OneDrive\Pictures\Saved Pictures\stock_detail.sql";

Enter the Product Number:101

Enter the Quantity Required:20

old  9: prdtNo:='&prdtNo';
new  9: prdtNo:='101';

old 10: Qty:='&Qty';
new 10: Qty:=      20;

Available Stock: 50

Product Name: computer

Values Updated

Available Stock: 30

```


Product Name: computer

PL/SQL procedure successfully completed.

SQL> @"C:\Users\divya\OneDrive\Pictures\Saved Pictures\stock_detail.sql";

Enter the Product Number:p101

Enter the Quantity Required:30

old 9: prdtNo:='&prdtNo';

new 9: prdtNo:='p101';

old 10: Qty:='&Qty';

new 10: Qty:= 30;

NO SUCH PRODUCT!

PL/SQL procedure successfully completed.

SQL> @"C:\Users\divya\OneDrive\Pictures\Saved Pictures\stock_detail.sql";

Enter the Product Number:103

Enter the Quantity Required:90

old 9: prdtNo:='&prdtNo';

new 9: prdtNo:='103';

old 10: Qty:='&Qty';

new 10: Qty:= 90;

Available Stock: 60

Product Name: cpu

NO SUFFICIENT STOCK!

PL/SQL procedure successfully completed.

Program 3:

Write a PL/SQL program to compute the selling price of books depending on the book code and category. Use Open, Fetch and Close. The Book_detail table contains columns: Book Code, Author, Title, Category and Price. Insert 10 records. The selling price=Price-Discout. The discount is calculated as follows:

The discount is calculated as follows:

Book Code	Category	Discount Percentage
A	Novels	10% of Price
	Technology	12.5% of Price
B	Commerce	18% of Price
	Science	19% of Price
C	Songs	25% of Price
	Sports	24% of Price
D	All	28% of Price

Print the result in tabular form with proper alignment

Book Code	category	title	author	price	discount %	discount amount	sell price
=====	=====	=====	=====	=====	=====	=====	=====

```
SQL> create table book_details(
```

```
    bkcode varchar2(2),
```

```
    author varchar2(20)not null,
```

```
    title varchar2(20)not null,
```

```
    category varchar2(20)not null,
```

```
    price number(10)not null);
```

Table created.

```
SQL> desc book_details;
```

Name	Null?	Type

BKCODE		VARCHAR2(2)
AUTHOR	NOT NULL	VARCHAR2(20)

TITLE	NOT NULL	VARCHAR2(20)
CATEGORY	NOT NULL	VARCHAR2(20)
PRICE	NOT NULL	NUMBER(10)

SQL> insert into book_details values('a','balaguruswamy','cprogam','technology',220);

1 row created.

SQL> insert into book_details values('a','swami','computernetwork','technology',230);

1 row created.

SQL> insert into book_details values('b','sharada','accounts','commerce',1200);

1 row created.

SQL> insert into book_details values('b','johnwilson','science','mathematics',1200);

1 row created.

SQL> insert into book_details values('c','balasubramanyam','devotional','songs',300);

1 row created.

SQL> insert into book_details values('c','viratkohli','sports spirit','sports',300);

1 row created.

SQL> insert into book_details values('d','viratkohli','sports spirit','sports',300);

1 row created.

SQL> insert into book_details values('d','shilpa','theteaderate','economics',700);

1 row created.

SQL> insert into book_details values('d','kruthi','businessworld','business',800);

1 row created.

SQL> select * from book_details;

BK	AUTHOR	TITLE	CATEGORY	PRICE
a	balaguruswamy	cprogam	technology	220
a	swami	computernetwork	technology	230
b	sharada	accounts	commerce	1200

b	johnwilson	science	mathematics	1200
c	balasubramanyam	devotional	songs	300
c	viratkohli	sports spirit	sports	300
d	viratkohli	sports spirit	sports	300
d	shilpa	theteaderate	economics	700
d	kruthi	businessworld	business	800

9 rows selected.

SQL> ed book1.sql

Set serveroutput ON;

DECLARE

bkCode varchar2(10);

bkAuthor varchar2(20);

bkTitle varchar2(20);

bkCategory varchar2(20);

bkDiscount number(10,1);

bkPrice number(10,1);

bkDisAmount number(10,1);

bkSell number(10,2);

CURSOR bk_details IS SELECT bkcode, author, title,category,price FROM book_details;

BEGIN

OPEN bk_details;

dbms_output.put_line(LPAD('-',150,'-'));dbms_output.put_line(RPAD('Book Code',20)||
RPAD('Category',20)||RPAD('Title',20)||RPAD('Author',20)||RPAD('Price',20)||RPAD('discou
nt %',20)||RPAD('discount amount',20)||RPAD('sell price',20));

dbms_output.put_line(LPAD('-',150,'-'));

LOOP

FETCH bk_details into bkCode,bkAuthor,bkTitle,bkCategory,bkPrice;

EXIT WHEN bk_details%notfound;

IF(bkCode='A' AND bkCategory='Novels') then

```
bkDiscount:=10;

ELSIF(bkCode='A' AND bkCategory='Technology') then

bkDiscount:=12.5;

ELSIF(bkCode='B' AND bkCategory='Commerce') then

bkDiscount:=18;

ELSIF(bkCode='B' AND bkCategory='Science') then

bkDiscount:=19;

ELSIF(bkCode='C' AND bkCategory='Songs') then

bkDiscount:=25;

ELSIF(bkCode='C' AND bkCategory='Sports') then

bkDiscount:=24;

ELSE

bkDiscount:=28;

END IF;

bkDisAmount:=bkPrice*bkDiscount/100;

bkSell:=bkPrice-bkDisAmount;

dbms_output.put_line(RPAD(bkCode,20)||RPAD(bkCategory,20)||RPAD(bkTitle,20)||RPAD(
bkAuthor,20)||RPAD(bkPrice,20)||RPAD(bkDiscount,20)||RPAD(bkDisAmount,20)||RPAD(b
kSell,20));

END LOOP;

dbms_output.put_line(LPAD('-',150,'-'));

CLOSE bk_details;

END;

/
```

OUTPUT:

SQL>set serveroutput ON;

SQL> @"C:\Users\divya\OneDrive\Pictures\Saved Pictures\book1.sql";

Book Code	Category	Title	Author	Price	discount %	discountAmount	sellprice
a	technology	cprogam	balaguruswamy	220	28	61.6	158.4
a	technology	computernetwork	swami	230	28	64.4	165.6
b	commerce	accounts	sharada	1200	28	336	864
b	mathematics	science	johnwilson	1200	28	336	864
c	songs	devotional	balasubramanyam	300	28	84	216
c	sports	sports spirit	viratkohli	300	28	84	216
d	sports	sports spirit	viratkohli	300	28	84	216
d	economics	theteaderate	Shilpa	700	28	196	504
d	business	businessworld	kruthi	800	28	224	576

PL/SQL procedure successfully completed.

Program 4:

Write a PL/SQL program to display employee pay bill (using Cursor For loop) Use a Procedure to receive basic pay and to compute DA, HRA, Tax, PF, Gross Pay and Net Pay(Use OUT). Base table contains the following columns empnum, empname, basic pay. Insert 3 records. Allowances are computed as follows

Basic Pay	DA	HRA
<=20000	35% of Basic	8% of Basic
>20000 & <=30000	38%	9%
>30000 & <=40000	40%	10%
>40000	45%	10%

Gross=Basic+DA+HRA

PF=12% of Gross or Rs. 2000 whichever is minimum.

PT=Rs. 100 upto Gross is 25,000 else Rs. 200.

Net=Gross-(PF+PT)

Print Pay slip as follows.

```
-----PAYSLIP-----
Empno      : 10011      Empname : Raj
Basic Pay   : 20000      P.F.    : 3432
DA          : 7000       P.T.    : 200
H.R.A.      : 1600
Gross       : 28600      Net Pay  : 24968
-----PAYSLIP-----
Empno      : 10012      Empname : Rani
Basic Pay   : 30000      P.F.    : 5292
DA          : 11400      P.T.    : 200
H.R.A.      : 2700
Gross       : 44100      Net Pay  : 38608
-----
```

SQL> connect system/tiger

Connected.

SQL> create table employee(

2 empnum varchar2(20)primary key,

3 empname varchar2(30)not null,

4 basicpay number(10,2)not null);

Table created.

SQL> desc employee;

Name	Null?	Type
EMPNUM	NOT NULL	VARCHAR2(20)
EMPNAME	NOT NULL	VARCHAR2(30)

BASICPAY NOT NULL NUMBER(10,2)

```
SQL> insert into employee values('e101','johnsmith',26000.78);
```

1 row created.

```
SQL> insert into employee values('e102','jerold wilson',34500.67);
```

1 row created.

```
SQL> insert into employee values('e103','ramachandra',44580.56);
```

1 row created.

```
SQL> insert into employee values('e104','anas',15580.39);
```

1 row created.

```
SQL> select * from employee;
```

EMPNUM	EMPNAME	BASICPAY
e101	johnsmith	26000.78
e102	jerold wilson	34500.67
e103	ramachandra	44580.56
e104	anas	15580.39

```
SQL> ed employee.sql
```

```
set serveroutput on;
```

```
declare
```

```
    cursor emp_cur is select empnum,empname,basicpay from employee;
```

```
    eno varchar2(20);
```

```
    name varchar2(30);
```

```
    bs number(10,2);
```

```
    da number(10,2);
```

```
    hra number(10,2);
```

```
    pt number(10,2);
```



```
    pf number(10,2);
    gross number(10,2);
    np number(10,2);
begin
    open emp_cur;
loop
    fetch emp_cur into eno,name,bs;
    exit when emp_cur%%notfound;
    if bs<=20000 then
        da:=bs*0.35;
        hra:=bs*0.8;
    elsif(bs>20000 and bs<=30000)then
        da:=bs*0.38;
        hra:=bs*0.9;
    elsif(bs>30000 and bs<=40000)then
        da:=bs*0.40;
        hra:=bs*0.10;
    elsif(bs>40000)then
        da:=bs*0.45;
        hra:=bs*0.10;
    end if;
    gross:=bs+da+hra;
    pf:=gross*0.12;
    if pf>=2000 then
        pf:=2000;
    end if;
    if gross<=25000 then
        pt:=100;
    else
```

```

        pt:=200;

    end if;

    np:=gross-(pf+pt);

    dbms_output.put_line( LPAD('=',30,'=')||'PAY SLIP'||LPAD('=',30,'='));

    dbms_output.put_line(RPAD('Emp no' ,10)||RPAD(eno,10)||RPAD('Emp name
:',10)||RPAD(name,10));

    dbms_output.put_line(RPAD('Basicpay' ,10)||RPAD(bs,10)||RPAD('PF
:',10)||RPAD(pf,10));

    dbms_output.put_line(RPAD('DA' ,10)||RPAD(da,10)||RPAD('PT
:',10)||RPAD(pt,10));

    dbms_output.put_line(RPAD('HRA' ,10)||RPAD(hra,10));

    dbms_output.put_line(RPAD('Gross' ,10)||RPAD(gross,10)||RPAD('Net
Pay',10)||RPAD(np,10));

    dbms_output.put_line( LPAD('=',70,'='));

end loop;

close emp_cur;

end;

/

```

OUTPUT:

```
SQL> set serveroutput ON;
```

```
SQL> @"C:\Users\divya\OneDrive\Pictures\Saved Pictures\employee.sql";
```

```

=====PAYSLIP=====

Empno   e101           Emp name :johnsmith
Basicpay 26000.78    PF :    2000
DA :    9880.3      PT :    200
HRA      23400.7
Gross :   59281.78   Net Pay : 57081.78
=====

```

=====PAYSLIP=====

Emp no e102 Emp name :jerold wil

Basicpay 34500.67 PF : 2000

DA : 13800.27 PT : 200

HRA : 3450.07

Gross : 51751.01 Net Pay : 49551.01

=====PAYSLIP=====

Emp no e103 Emp name :ramachandr

Basicpay 44580.56 PF : 2000

DA : 20061.25 PT : 200

HRA 4458.06

Gross : 69099.87 Net Pay : 66899.87

=====PAYSLIP=====

Emp no e104 Emp name :anas

Basicpay 15580.39 PF : 2000

DA : 5453.14 PT : 200

HRA 12464.31

Gross : 33497.84 Net Pay : 31297.84

PL/SQL procedure successfully completed.

Program 5:

Given the following tables: ITEM_MASTER (item no, name, stock, unit price) [Apply the Primary key and check constraint for stock and price as >0] [Insert 5 records] ITEM_TRANS (item no, quantity and trans date) Create a package PCK_ITEM includes a function CHK_ITEM and a procedure PROC_ITEM. Function CHK_ITEM gets one argument item no and is used to check whether the parameter item no exists in ITEM_MASTER and should return 1 if exist. Otherwise, 0 and displays proper message. Procedure PROC_ITEM gets two arguments item no and quantity, and is used to perform the following if item exists. If required quantity is not available, give appropriate message. If available, insert a record of this transaction to ITEM_TRANS and modify the stock in ITEM_MASTER. Write a PL/SQL program to accept ITEM_NO and Quantity needed of required item. Use Package to do the transaction process (Transaction date can be current date). OUTPUT to be shown as follows:

```
Enter value for accept_itemno: 1
old 5: X:=&accept_itemno;
new 5: X:=1;
Enter value for quantity: 3
old 6: M:=&quantity;
new 6: M:=3;
Item :aa Quantity :3 Price :15 Total Amount :45
```

SQL> connect system/tiger

Connected.

SQL> create table item_master(

- 2 itemno number(5),
- 3 name varchar2(10),
- 4 stock number(5),
- 5 unit_price number(5),
- 6 constraint stock_price check(stock>0 and unit_price>0));

Table created.

```
SQL> create table item_trans(
```

```
2 itemno number(10),
```

```
3 qty number(5),
```

```
4 tdate date);
```

Table created.

```
SQL> desc item_master;
```

Name	Null?	Type

ITEMNO		NUMBER(5)
NAME		VARCHAR2(10)
STOCK		NUMBER(5)
UNIT_PRICE		NUMBER(5)

```
SQL> desc item_trans;
```

Name	Null?	Type

ITEMNO		NUMBER(10)
QTY		NUMBER(5)
TDATE		DATE

```
SQL> insert into item_master values(101,'printer',800,200);
```

1 row created.

```
SQL> insert into item_master values(102,'keyboard',700,100);
```

1 row created.

```
SQL> insert into item_master values(103,'mouse',300,1500);
```

1 row created.

```
SQL> insert into item_master values(104,'pendrive',100,150);
```

1 row created.

```
SQL> select * from item_master;
```

ITEMNO	NAME	STOCK	UNIT_PRICE
101	printer	800	200
102	keyboard	700	100
103	mouse	300	1500
104	pendrive	100	150

```
SQL> ed pck_item.sql
```

```
create or replace package pck_item as
```

```
function ch_item( n number ) return number;
```

```
procedure proc_item(n number,q number);
```

```
end pck_item;
```

```
/
```

```
SQL> ed pck_body.sql
```

```
create or replace package body pck_item as
```

```
function ch_item(n in number) return number is
```

```
ino number(5);
```

```
begin
```

```
    select itemno into ino from item_master where itemno=n;
```

```
    return 1;
```

```
    exception when no_data_found then
```

```
        return 0;
```

```
end ;
```

```
procedure proc_item(n in number,q in number) is
```

```
ino number(5);
```

```
iname varchar(10);
```

```
stk number(10);
```

```
price number(5);
```

```

totamt number(10);

begin
    if(pck_item.ch_item(n)=1) then
        select itemno,name,stock,unit_price into ino,iname,stk,price from item_master
where itemno=n;
        if(stk<q) then
            dbms_output.put_line('required stock not available');
        else
            insert into item_trans values(ino,q,sysdate);
            update item_master set stock=stock-q where itemno=n;
            totamt:=q*price;
            dbms_output.put_line(RPAD('Item:',10)||RPAD(iname,10)||RPAD('Quantity:',10)||RP
AD(q,10)||RPAD('Price:',10)||RPAD(price,10)||RPAD('Total:',10)||RPAD(totamt,10));
        end if;
    end if;
end;
end pck_item;
/

```

SQL> ed prock_item.sql

```

set serveroutput on;

prompt Enter Item Number;;

accept ino;

prompt Enter Quantity;;

accept qty;

begin
if(pck_item.ch_item(&ino)=1) then
    dbms_output.put_line('Item Exists. ');
else
    dbms_output.put_line('Item Does Not Exists. ');
end if;

```

```
        pck_item.proc_item(&ino,&qty);  
end;  
/  

```

OUTPUT:

SQL> set serveroutput ON;

Sql>@"C:\Users\divya\OneDrive\Pictures\Screenshots\pck_item.sql";

Package created.

SQL> @"C:\Users\divya\OneDrive\Pictures\Saved Pictures\pck_body.sql";

Package body created.

SQL> @"C:\Users\divya\OneDrive\Pictures\Saved Pictures\prock_item.sql";

Enter Item Number:

101

Enter Quantity:

5

old 2: if(pck_item.ch_item(&ino)=1) then

new 2: if(pck_item.ch_item(101)=1) then

old 7: pck_item.proc_item(&ino,&qty);

new 7: pck_item.proc_item(101,5);

Item Exists.

Item : printer Quantity :5 Price : 200 Total : 1000

PL/SQL procedure successfully completed.

SQL> @"C:\Users\divya\OneDrive\Pictures\Saved Pictures\prock_item.sql";

Enter Item Number:

106

Enter Quantity:

4

old 2: if(pck_item.ch_item(&ino)=1) then


```
new 2: if(pck_item.ch_item(106)=1) then  
old 7:    pck_item.proc_item(&ino,&qty);  
new 7:    pck_item.proc_item(106,4);
```

Item Does Not Exists.

PL/SQL procedure successfully completed.

Program 6:

Create a package which includes a function to compute the factorial of a number, a procedure to compute the value of nCr , and another procedure to compute nPr both uses the factorial function. Execute the package program for the required calculation.

Pck_math.sql

```
CREATE OR REPLACE PACKAGE pck_math AS  
  
    FUNCTION fact(n NUMBER) RETURN NUMBER;  
  
    PROCEDURE nCr(n NUMBER, r NUMBER);  
  
    PROCEDURE nPr(n NUMBER, r NUMBER);  
  
END pck_math;  
  
/
```

Create the Package Body

Pck_bodyy.sql

```
CREATE OR REPLACE PACKAGE BODY pck_math AS  
  
    -- Function to compute factorial  
  
    FUNCTION fact(n NUMBER) RETURN NUMBER IS  
  
        res NUMBER := 1;  
  
    BEGIN  
  
        IF n < 0 THEN  
  
            RETURN NULL; -- factorial not defined for negative numbers  
  
        ELSIF n = 0 THEN  
  
            RETURN 1;  
  
        ELSE  
  
            FOR i IN 1..n LOOP  
  
                res := res * i;  
  
            END LOOP;  
  
            RETURN res;  
  
        END IF;  
  
    END fact;  
  
    PROCEDURE nCr(n NUMBER, r NUMBER) IS  
  
        fact_n NUMBER;  
        fact_r NUMBER;  
        res NUMBER;  
  
    BEGIN  
  
        fact_n := fact(n);  
        fact_r := fact(r);  
        res := fact_n / fact_r;  
  
        RETURN res;  
  
    END nCr;  
  
    PROCEDURE nPr(n NUMBER, r NUMBER) IS  
  
        fact_n NUMBER;  
        fact_r NUMBER;  
        res NUMBER;  
  
    BEGIN  
  
        fact_n := fact(n);  
        fact_r := fact(n - r + 1);  
        res := fact_n / fact_r;  
  
        RETURN res;  
  
    END nPr;  
  
END pck_math_body;
```

```

    END IF;

END fact;

-- Procedure to compute nCr

PROCEDURE nCr(n NUMBER, r NUMBER) IS

    result NUMBER;

BEGIN

    IF r > n OR n < 0 OR r < 0 THEN

        DBMS_OUTPUT.PUT_LINE('Invalid inputs for nCr.');
```

ELSE

```

        result := fact(n) / (fact(r) * fact(n - r));

        DBMS_OUTPUT.PUT_LINE('nCr(' || n || ', ' || r || ') = ' || result);

    END IF;

END nCr;

-- Procedure to compute nPr

PROCEDURE nPr(n NUMBER, r NUMBER) IS

    result NUMBER;

BEGIN

    IF r > n OR n < 0 OR r < 0 THEN

        DBMS_OUTPUT.PUT_LINE('Invalid inputs for nPr.');
```

ELSE

```

        result := fact(n) / fact(n - r);

        DBMS_OUTPUT.PUT_LINE('nPr(' || n || ', ' || r || ') = ' || result);

    END IF;

END nPr;

END pck_math;

/
```

OUTPUT:

SQL> connect system/tiger

Connected.

SQL> @"C:\Users\divya\OneDrive\Pictures\Saved Pictures\pck_math.sql";

Package created.

SQL> @"C:\Users\divya\OneDrive\Pictures\Saved Pictures\pck_bodyy.sql";

Package body created.

SQL> set serveroutput ON;

SQL> BEGIN

 pck_math.nCr(5, 2);

 END;

 /

nCr(5, 2) = 10

PL/SQL procedure successfully completed.

SQL> BEGIN

 pck_math.nPr(5, 2);

 END;

 /

nPr(5, 2) = 20

PL/SQL procedure successfully completed.

Program 7:

Create a trigger to update the MASTER table when a record is inserted into SALES table and create another trigger to update the MASTER table when a record is inserted or updated or deleted in NEWSTOCK table. Assume the suitable columns for all the tables.

SQL> create table MASTER(

2 PID number(10)primary key,

3 pname varchar(10)not null,

4 total_stock number(7),

5 total_sales number(9));

Table created.

SQL> desc MASTER;

Name	Null?	Type

PID	NOT NULL	NUMBER(10)
PNAME	NOT NULL	VARCHAR2(10)
TOTAL_STOCK		NUMBER(7)
TOTAL_SALES		NUMBER(9)

SQL> insert into MASTER values(101,'MOUSE',100,0);

1 row created.

SQL> insert into MASTER values(102,'DVD',70,0);

1 row created.

SQL> insert into MASTER values(103,'CD',90,0);

1 row created.

SQL> insert into MASTER values(104,'KEYBOARD',150,0);

1 row created.

```
SQL> insert into MASTER values(105,'MONITOR',50,0);
```

1 row created.

```
SQL> select*from MASTER;
```

PID	PNAME	TOTAL_STOCK	TOTAL_SALES
101	MOUSE	100	0
102	DVD	70	0
103	CD	90	0
104	KEYBOARD	150	0
105	MONITOR	50	0

```
SQL> create table SALES (
```

```
2  SID varchar(10)primary key,
```

```
3  PID number(10),
```

```
4  sales_qty number(20));
```

Table created.

```
SQL> desc SALES;
```

Name	Null?	Type
SID	NOT NULL	VARCHAR2(10)
PID		NUMBER(10)
SALES_QTY		NUMBER(20)

```
SQL> create table NEWSTOCK (
```

```
2  stock_id number(10) primary key,
```

```
3  PID number(10),
```

```
4  stock_qty number(10));
```

Table created.

```
SQL> desc NEWSTOCK;
```

Name	Null?	Type
STOCK_ID	NOT NULL	NUMBER(10)
PID		NUMBER(10)
STOCK_QTY		NUMBER(10)

```
SQL> ed SALES.sql
```

```
CREATE OR REPLACE TRIGGER update_master_after_sales
after insert on SALES
for each row
begin
    update MASTER set total_stock=total_stock-:NEW.sales_qty,
    total_sales=total_sales+:NEW.sales_qty where PID=:NEW.PID;
End;
/
```

```
SQL> ed NEWSTOCK.sql
```

```
CREATE OR REPLACE TRIGGER update_master_after_newstock
after insert or update or delete on NEWSTOCK
for each row
begin
    IF inserting then
        update MASTER set total_stock=total_stock+:NEW.stock_qty where
        PID=:NEW.PID;
    ELSIF updating then
        update MASTER set total_stock=total_stock+(:NEW.stock_qty-:OLD.stock_qty)
        where PID=:NEW.PID;
    ELSIF deleting then
        update MASTER set total_stock=total_stock-:OLD.stock_qty where PID=:OLD.PID;
    END IF;
```

END;

/

SQL> set serveroutput ON;

SQL> @"C:\Users\divya\OneDrive\Pictures\Saved Pictures\SALES.sql";

Trigger created.

SQL> insert into SALES values('S1',101,20);

1 row created.

SQL> insert into SALES values('S2',102,10);

1 row created.

SQL> insert into SALES values('S3',103,30);

1 row created.

SQL> select*from SALES;

SID	PID	SALES_QTY
-----	-----	-----------

S1	101	20
----	-----	----

S2	102	10
----	-----	----

S3	103	30
----	-----	----

SQL> select*from MASTER;

PID	PNAME	TOTAL_STOCK	TOTAL_SALES
-----	-------	-------------	-------------

101	MOUSE	80	20
-----	-------	----	----

102	DVD	60	10
-----	-----	----	----

103	CD	60	30
-----	----	----	----

104	KEYBOARD	150	0
-----	----------	-----	---

105	MONITOR	50	0
-----	---------	----	---


```
SQL> @"C:\Users\divya\OneDrive\Pictures\Saved Pictures\NEWSTOCK.sql";
```

Trigger created.

```
SQL> insert into NEWSTOCK values(201,101,60);
```

1 row created.

```
SQL> insert into NEWSTOCK values(202,102,40);
```

1 row created.

```
SQL> insert into NEWSTOCK values(203,103,55);
```

1 row created.

```
SQL> select*from NEWSTOCK;
```

STOCK_ID	PID	STOCK_QTY
----------	-----	-----------

201	101	60
-----	-----	----

202	102	40
-----	-----	----

203	103	55
-----	-----	----

```
SQL> select*from MASTER;
```

PID	PNAME	TOTAL_STOCK	TOTAL_SALES
-----	-------	-------------	-------------

101	MOUSE	140	20
-----	-------	-----	----

102	DVD	100	10
-----	-----	-----	----

103	CD	115	30
-----	----	-----	----

104	KEYBOARD	150	0
-----	----------	-----	---

105	MONITOR	50	0
-----	---------	----	---

```
SQL> update NEWSTOCK
```

```
2 set stock_qty=80
```

```
3 where stock_id=201;
```

1 row updated.

```
SQL> update NEWSTOCK
```

```
2   set stock_qty=70
```

```
3   where stock_id=202;
```

```
1 row updated.
```

```
SQL> select*from NEWSTOCK;
```

```
STOCK_ID  PID STOCK_QTY
```

```
-----
```

```
201      101      80
```

```
202      102      70
```

```
203      103      55
```

```
SQL> select*from MASTER;
```

```
PID PNAME  TOTAL_STOCK TOTAL_SALES
```

```
101 MOUSE    160         20
```

```
102 DVD      130         10
```

```
103 CD       115         30
```

```
104 KEYBOARD 150          0
```

```
105 MONITOR  50          0
```

```
SQL> delete from NEWSTOCK where stock_id=201;
```

```
1 row deleted.
```

```
SQL> select*from MASTER;
```

```
PID PNAME  TOTAL_STOCK TOTAL_SALES
```

```
101 MOUSE    80         20
```

```
102 DVD      130         10
```

```
103 CD       115         30
```

```
104 KEYBOARD 150          0
```

```
105 MONITOR  50          0
```